Functions and modules

Functions: Block of code that executes only when it is explicitly called.

use "def" keyword → defining a function

```
def clean_name(name): → begins function body
    return name.strip().title()
```

print(clean_name(" priya ")) => Priya

Arguments types

1. Position argument: assign based on position

Keyword argument: explicity identify parameter

Ex: add(a=3, b=5)

Default argument: default value on creating parameter

*args → accepts multiple positional arguments

```
Ex: def add(*args):
        return sum(args)
```

dynamically have no. of args

```
Print(1,2)
Print(1,2,3,4)
```
→ both coorks

→ collects extra positional args into a tuple

name(args)is conventions, * matters

**kwargs (keycoord args) → accept multiple kwargs

```
Ex: def show_name(**kwargs):
        print(kwargs)

show name(name="test", city="cbe")
```

Lambda functions

→ Small anonymous function written as Single expression with lambda keyword

Syntax: lambda arguments : expression

→ no def or no return

Ex: def square_num(x): normal
    return x*x

square_lambda = lambda x : x*x (lambda)
↳ keyword ↳ expression

Built in lambda function,
list() : materialize their result
map() : transform all items

num = [1, 2, 3, 4]

double = list(map(lambda x: x*2, num))

filter() : Selected items

sorted() : custom sorting

Ex: students = [("Arun", 21), ("Priya", 19), ("Arjun", 23)]

Sort by their age list with tuple

Sorted_students = sorted(students,
key = lambda student: student[1])

Strings:

methods : strip() , lower() , upper() , replace() , split()

join() → "|".join(names)

slice() → (start : stop : step)

Zip() → Process multiple sequence

Ex: names = ["Arun", "Billa", "Chad"], marks = [99, 80, 85]

for name, mark in zip(names, marks):
    print(name, mark)

Exceptional handling:

try, except,
↳ catches exception

else → if no exception occurs it run

finally → always runs

Ex:
```
try:
    #statem
except (___):
finally:
```

raise : intentionally trigger exception

logging: Built in module

five levels

Debug: Detailed info during development

Info: Normal milestone, like starting pipeline

Warning: Something unexpected occur, program continue

Error: An Operational failed

Critical: A severe problem threatens continued operation

Ex:
```
logging.basicConfig(
        level=logging.DEBUG,
        format="%(levelname)s:%(message)s")

logging.debug("checking)
    info("Started")
    warning("records values are missing")
    error("failed to process one order")
    Critical("Pipeline can not continue")
```

Diff from Print(): Designed for recording events

Debugging (pdb & breakpoint)

breakpoint(): stop executing at that line
and open interactive terminal

Debugger Commands:

n(next): execute current line and move to next

s(step): Step inside the function called on current line

c(continue): Resume program

p variable name: (print)

q(quit): Exit

Advanced Python

Iterators: Traverse through all the elements in collections (list, tuple or dict) 1 at a time

iter() → returns iterator obj

next() → returns next item in sequence

Ex. num = [10, 20] iterator = iter(num)

print (next (iterator)) → 10
next (iterator)) → 20
next (iterator)) → StopIteration

Generators: Simpler, more memory efficient way to create iterators

yield → pause execution, send back value and save its entire state (variable, loop state, line number)

Ex: def simple_generator()
print ("First")
yield "A"
Print ("Second")
yield "B"

gen = simple_generator()
print (next (gen));
print (next (gen));